Графики и таблицы в Python

Matplotlib, Seaborn и pandas: код и его результат

Школа 1514

Занятие 02

Matplotlib: начинаем с графика

Matplotlib — библиотека Python для построения графиков. Она умеет рисовать линии, точки, столбцы и оси: нам не нужно программировать их отрисовку самим.

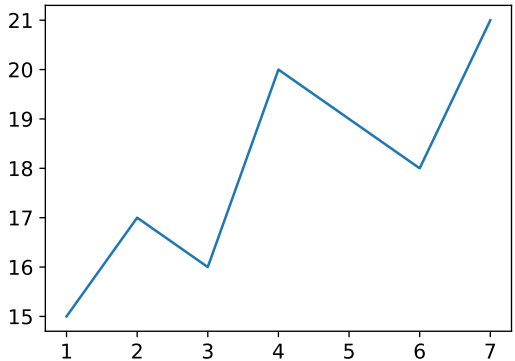
Начнём с модуля pyplot. Его функции работают с текущим рисунком: построить, подписать, показать. Переменные fig и ax для первых примеров не нужны.

На слайдах код находится слева, а справа — результат выполнения этого же кода. Примеры идут по порядку: созданные переменные используются дальше.

Первый график: температура за неделю

```
import matplotlib.pyplot as plt

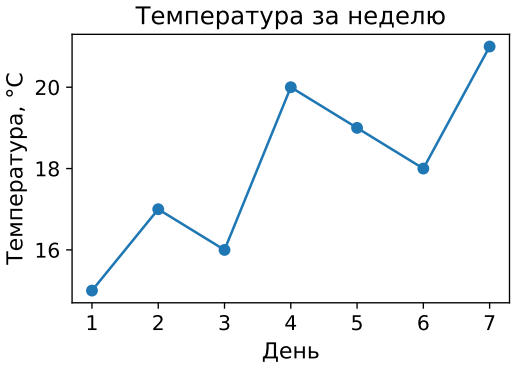
days = [1, 2, 3, 4, 5, 6, 7]
temperature = [15, 17, 16, 20, 19, 18, 21]
plt.plot(days, temperature)
plt.show()
```



Первый список задаёт x, второй — y. Видно, как температура менялась по дням; но без подписей посторонний читатель ещё не знает, что означают числа.

Добавляем смысл: оси, заголовок и точки

```
plt.plot(days, temperature, marker="o")
plt.xlabel("День")
plt.ylabel("Температура, °C")
plt.title("Температура за неделю")
plt.show()
```

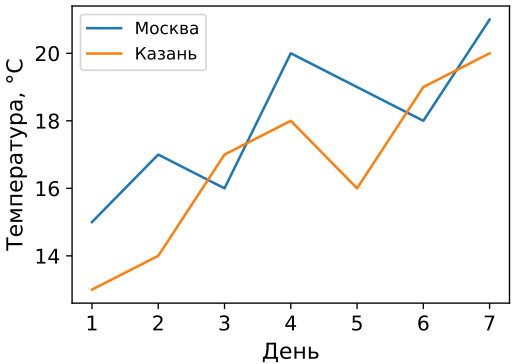


Кружки — измерения, линия помогает проследить изменение между соседними днями.

Попробуйте заменить температуру седьмого дня на 16 и предскажите изменение графика.

Два города на одном рисунке

```
kazan = [13, 14, 17, 18, 16, 19, 20]
plt.plot(days, temperature,
         label="Москва")
plt.plot(days, kazan, label="Казань")
plt.xlabel("День")
plt.ylabel("Температура, °C")
plt.legend()
plt.show()
```

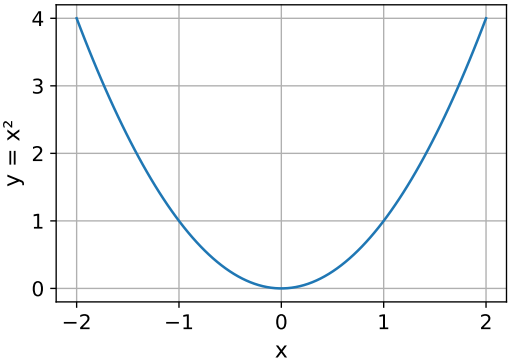


В четвёртый день в Москве 20 °C, в Казани 18 °C. Линии уместны: дни упорядочены, и соседство наблюдений имеет смысл.

График математической функции

```
import numpy as np

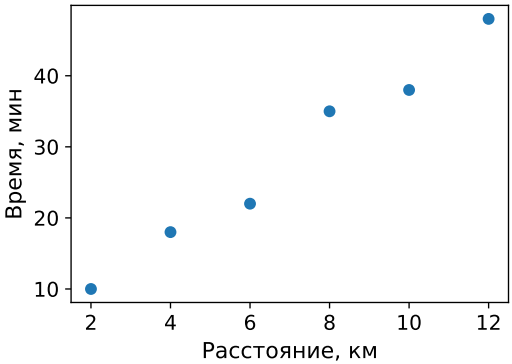
x = np.linspace(-2, 2, 100)
plt.plot(x, x ** 2)
plt.xlabel("x")
plt.ylabel("y = x²")
plt.grid(True)
plt.show()
```



Это та же команда plot, но данные не измерены, а вычислены. grid добавляет сетку. Попробуйте заменить x ** 2 на np.sin(x).

Scatter: расстояние и время поездки

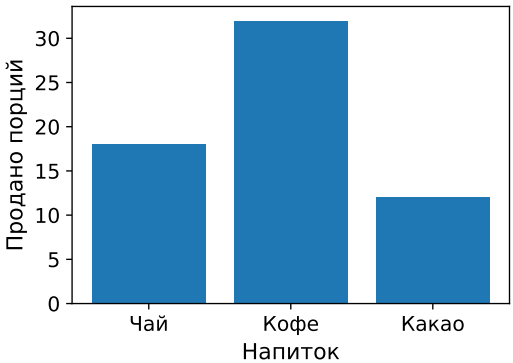
```
distance = [2, 4, 6, 8, 10, 12]
minutes = [10, 18, 22, 35, 38, 48]
plt.scatter(distance, minutes)
plt.xlabel("Расстояние, км")
plt.ylabel("Время, мин")
plt.show()
```



Большему расстоянию обычно соответствует большее время. Отдельные точки позволяют видеть отклонения, а не только общее направление связи.

Bar: сравниваем продажи напитков

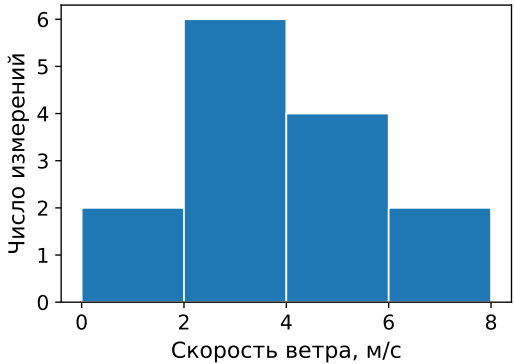
```
drinks = ["Чай", "Кофе", "Какао"]
cups = [18, 32, 12]
plt.bar(drinks, cups)
plt.xlabel("Напиток")
plt.ylabel("Продано порций")
plt.show()
```



Кофе продали 32 порции, чая — 18. Столбцы подходят для категорий; соединять напитки линией обычно не имеет смысла.

Hist: как распределена скорость ветра

```
wind = [1, 3, 2, 5, 4, 2, 6,
        3, 1, 4, 7, 5, 2, 3]
plt.hist(wind, bins=[0, 2, 4, 6, 8],
         edgecolor="white")
plt.xlabel("Скорость ветра, м/с")
plt.ylabel("Число измерений")
plt.show()
```



В [2, 4) шесть измерений: три двойки и три тройки. Число 4 относится к [4, 6). Только последний интервал включает правую границу.

Pandas: зачем хранить данные в таблице

В кафе или магазине у каждой позиции есть название, категория, цена и число продаж. Если хранить всё отдельными списками, при сортировке легко потерять связь между товаром и его ценой. Таблица хранит эти значения вместе.

pandas — библиотека Python для работы с таблицами. Она позволяет читать файлы, выбирать строки, считать по столбцам и получать сводки. Основной объект, DataFrame, — таблица с именованными столбцами. Начнём с четырёх товаров, чтобы каждый результат можно было проверить вручную.

Создадим таблицу небольшого магазина

```
import pandas as pd

table = pd.DataFrame({
    "item": ["Тетрадь", "Блокнот",
             "Ручка", "Карандаш"],
    "category": ["Бумага", "Бумага",
                 "Письмо", "Письмо"],
    "price": [80, 120, 30, 20],
    "sold": [10, 5, 20, 15],
})
table
```

	item	category	price	sold
0	Тетрадь	Бумага	80	10
1	Блокнот	Бумага	120	5
2	Ручка	Письмо	30	20
3	Карандаш	Письмо	20	15

pd — короткое имя pandas, ключи словаря стали названиями столбцов. Первая строка: тетрадь стоит 80 рублей, продано 10 штук. Числа 0–3 слева — индекс: метки строк, а не ещё один признак товара.

Размер таблицы и названия столбцов

```
print(table.shape)
print(len(table))
print(table.columns.tolist())
```

```
(4, 4)
4
['item', 'category', 'price', 'sold']
```

Получили четыре строки и четыре столбца. Эти команды только смотрят на таблицу и ничего из неё не удаляют.

Выбираем один столбец

```
prices = table["price"]
prices
```

	price
0	80
1	120
2	30
3	20

Справа показаны индекс и значения выбранной Series. В Jupyter дополнительно появляется строка Name: price, dtype: int64 — имя столбца и тип его значений.

Вычисляем по столбцу

```
print("Средняя цена:", prices.mean())
print("Минимум:", prices.min())
print("Максимум:", prices.max())
print("Всего продано:",
      table["sold"].sum())
```

```
Средняя цена: 62.5
Минимум: 20
Максимум: 120
Всего продано: 50
```

Средняя цена: (80 + 120 + 30 + 20) / 4 = 62.5 рубля. Средняя цена товара здесь не взвешена по числу продаж.

Выбираем несколько столбцов

```
table[["item", "sold"]]
```

	item	sold
0	Тетрадь	10
1	Блокнот	5
2	Ручка	20
3	Карандаш	15

Теперь удобно смотреть только названия и количество продаж. Попробуйте вместо sold выбрать price и объяснить, что изменится.

Условие сначала даёт ответы True или False

```
mask = table["price"] >= 80
mask
```

	price
0	True
1	True
2	False
3	False

Тетрадь и блокнот подходят: напротив них True. Ручка и карандаш не подходят: напротив них False.

Используем маску для отбора строк

```
expensive = table[mask]
expensive
```

	item	category	price	sold
0	Тетрадь	Бумага	80	10
1	Блокнот	Бумага	120	5

Остались два товара, а не два отдельных числа. Индексы сохранились из исходной таблицы.

Попробуйте отобрать товары, которых продано больше 10 штук.

Два условия одновременно

```
mask = ((table["category"] == "Письмо")
        & (table["sold"] >= 18))
table[mask]
```

	item	category	price	sold
2	Ручка	Письмо	30	20

В категории «Письмо» два товара, но не меньше 18 штук продано только ручек. Подходящая строка одна, и её индекс по-прежнему 2.

Упорядочим товары по продажам

```
popular = table.sort_values(
    "sold", ascending=False
)
popular
```

	item	category	price	sold
2	Ручка	Письмо	30	20
3	Карандаш	Письмо	20	15
0	Тетрадь	Бумага	80	10
1	Блокнот	Бумага	120	5

Первыми идут ручки: продано 20 штук. Исходная table не меняет порядок. popular.head(2) покажет два наиболее продаваемых товара.

Новый столбец: выручка по каждому товару

```
table["revenue"] = (
    table["price"] * table["sold"]
)
table[["item", "revenue"]]
```

	item	revenue
0	Тетрадь	800
1	Блокнот	600
2	Ручка	600
3	Карандаш	300

Это выручка, не прибыль: расходы нам неизвестны. Справа для наглядности выведены только название и новый столбец.

Groupby: отдельная сумма для каждой категории

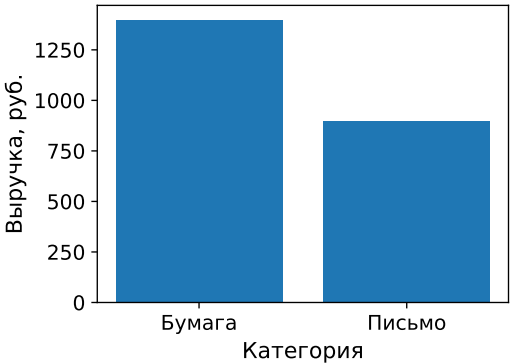
```
groups = table.groupby("category")
revenue_by_category = (
    groups["revenue"].sum()
)
revenue_by_category
```

category	revenue
Бумага	1400
Письмо	900

В результате индекс — названия категорий, значения — их выручка. Получились 1400 и 900 рублей. Сам groupby ещё не считает: действие задаёт sum.

Покажем готовую сводку на графике

```
plt.bar(revenue_by_category.index,
        revenue_by_category.values)
plt.xlabel("Категория")
plt.ylabel("Выручка, руб.")
plt.show()
```



Высота столбца — выручка категории, а не количество товаров. Подпись оси y помогает не перепутать смысл агрегированных чисел.

Pandas: читаем данные о поездках

Таблицу не обязательно создавать вручную. CSV — текстовый формат, где строки соответствуют наблюдениям, а значения разделены запятыми. Первая строка задаёт названия столбцов. read_csv превращает такой файл в обычный DataFrame.

В приложенном rides.csv одна строка — одна поездка на велосипеде. trip_id — номер, bike — тип велосипеда, distance_km — километры, minutes — время. Данные синтетические и не описывают реальных людей. После загрузки работают те же столбцы, маски и группировки, что и в маленьком примере магазина.

Загрузим файл и посмотрим начало

```
raw = pd.read_csv(DATA / "rides.csv")
raw.head(4)
```

	trip_id	bike	distance_km	minutes
0	1	Городской	14.0	58.9
1	2	Горный	15.8	78.8
2	3	Шоссейный	23.1	72.1
3	4	Городской	23.3	98.5

Одна поездка имеет и расстояние, и время, и тип велосипеда. Первые строки полезно прочитать словами перед тем, как считать статистики.

Размер и неизвестные значения

```
print("Размер:", raw.shape)
print("Пропуски:")
print(raw.isna().sum())
```

```
Размер: (61, 4)
Пропуски:
trip_id        0
bike           0
distance_km    2
minutes        3
dtype: int64
```

Всего 61 строка. Не записаны два расстояния и три длительности. Заменять их нулём означало бы придумать сведения, которых нет.

Удалим технический повтор и подготовим пары

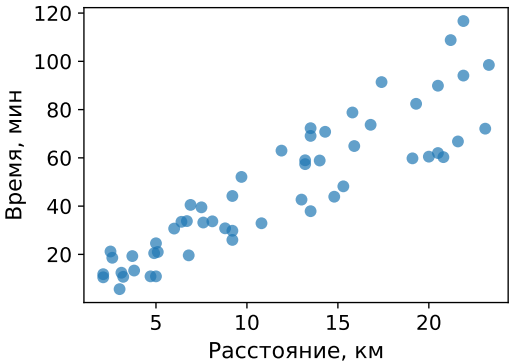
```
clean = raw.drop_duplicates().copy()
trips = clean.dropna(
    subset=["distance_km", "minutes"]
)
print("Без повторов:", len(clean))
print("Полных пар:", len(trips))
```

```
Без повторов: 60
Полных пар: 55
```

Осталось 60 разных поездок и 55 полных пар. В других наборах одинаковые строки могут быть разными событиями: удалять повторы без понимания их смысла нельзя.

Столбцы вместо списков в Matplotlib

```
plt.scatter(trips["distance_km"],
            trips["minutes"], alpha=0.7)
plt.xlabel("Расстояние, км")
plt.ylabel("Время, мин")
plt.show()
```



На рисунке 55 точек. Часть разброса связана с разной скоростью велосипедов: одного расстояния недостаточно, чтобы точно предсказать время.

Seaborn: связываем график с таблицей

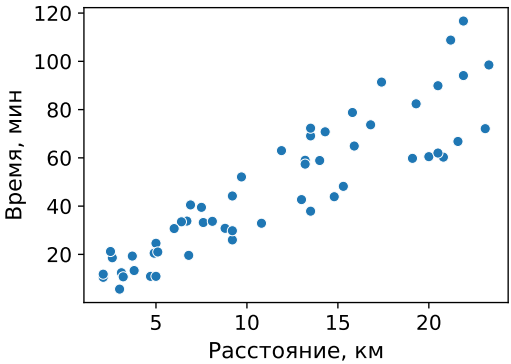
Seaborn — библиотека поверх Matplotlib, которая упрощает распространённые графики анализа данных. Ей можно передать всю таблицу и назвать столбцы для x, y и цвета. Она сама найдёт категории, назначит цвета и создаст легенду.

Сначала построим ту же диаграмму рассеяния. Затем добавим информацию о типе велосипеда. Подписи через plt продолжат работать: основой рисунка остаётся Matplotlib. Переменная ax и здесь не обязательна.

Тот же график через Seaborn

```
import seaborn as sns

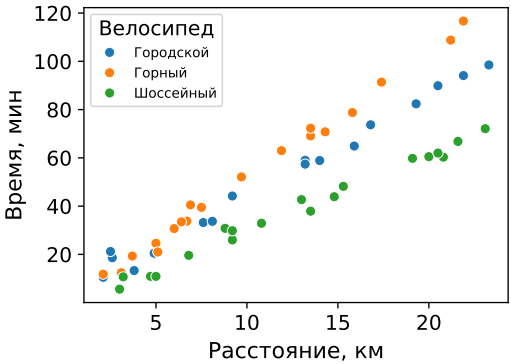
sns.scatterplot(data=trips,
    x="distance_km", y="minutes")
plt.xlabel("Расстояние, км")
plt.ylabel("Время, мин")
plt.show()
```



Координаты совпадают с предыдущим примером. Seaborn не добавляет новые поездки и не усредняет данные в scatterplot.

Цвет показывает тип велосипеда

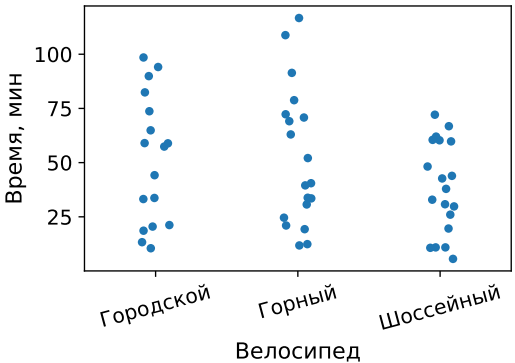
```
sns.scatterplot(data=trips,
    x="distance_km", y="minutes",
    hue="bike")
plt.xlabel("Расстояние, км")
plt.ylabel("Время, мин")
plt.legend(title="Велосипед", fontsize=8)
plt.show()
```



hue="bike"задаёт цвет по типу велосипеда и создаёт легенду. Разные скорости здесь заложены в генераторе: это учебная модель, а не реальное измерение преимуществ велосипеда.

Посмотрим на отдельные поездки в категориях

```
np.random.seed(1514)
sns.stripplot(data=trips,
    x="bike", y="minutes", jitter=True)
plt.xlabel("Велосипед")
plt.ylabel("Время, мин")
plt.xticks(rotation=15)
plt.show()
```



seed фиксирует случайное горизонтальное смещение точек. Время внутри категории различается, ведь расстояния тоже разные. Сравнивать скорость только по этому рисунку было бы неправильно.

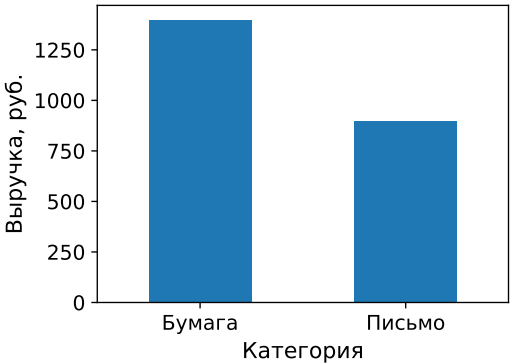
Встроенные графики pandas

После Matplotlib и Seaborn познакомимся ещё с одной записью: Series.plot() и DataFrame.plot(). Эти методы позволяют построить распространённый график прямо из столбца или таблицы. По умолчанию pandas использует Matplotlib, поэтому знакомые подписи и сохранение через plt продолжают работать.

kind задаёт тип: bar — столбцы, hist — гистограмма, scatter — точки, line — линии. Для Series подписи часто берутся из индекса, значения — из содержимого столбца. Для DataFrame мы явно укажем x и y. Создавать fig и ax для этих примеров не нужно.

Series.plot: готовая выручка в виде столбцов

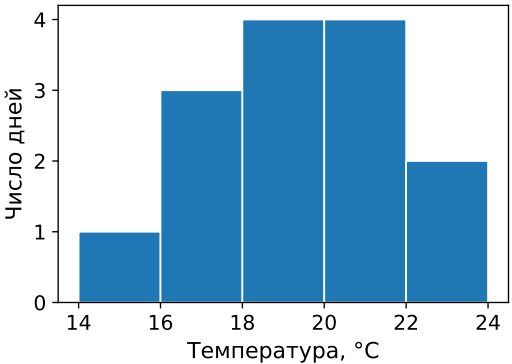
```
revenue_by_category.plot(
    kind="bar", rot=0
)
plt.xlabel("Категория")
plt.ylabel("Выручка, руб.")
plt.show()
```



Это тот же результат, что у plt.bar для сводки: 1400 и 900 рублей. rot=0 оставляет подписи горизонтальными; отдельный plt.figure не нужен.

Гистограмма столбца из другого CSV

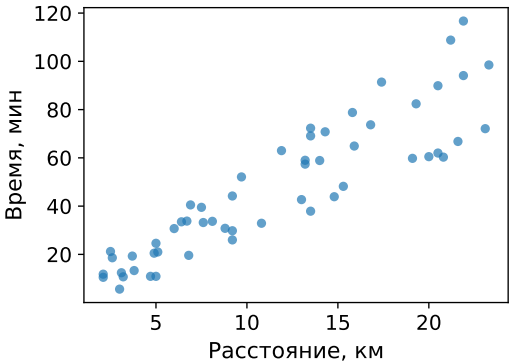
```
weather = pd.read_csv(
    DATA / "weather.csv"
)
weather["moscow"].plot(
    kind="hist",
    bins=[14, 16, 18, 20, 22, 24],
    edgecolor="white"
)
plt.xlabel("Температура, °C")
plt.ylabel("Число дней")
plt.show()
```



Всего 14 наблюдений. Высота прямоугольника сообщает, сколько дней температура лежала в данном интервале, а не в какой день она наблюдалась.

DataFrame.plot: явно выбираем два столбца

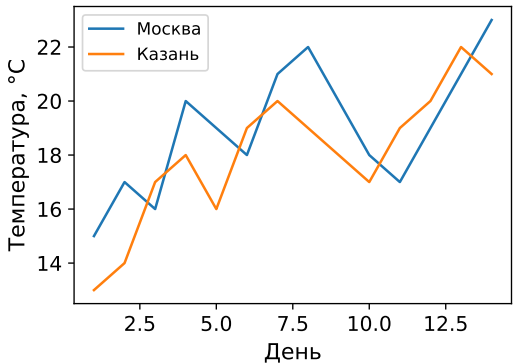
```
trips.plot(kind="scatter",
    x="distance_km", y="minutes",
    alpha=0.7)
plt.xlabel("Расстояние, км")
plt.ylabel("Время, мин")
plt.show()
```



Снова те же 55 пар. Для быстрого графика этого достаточно; для удобного цвета по категориям можно вернуться к Seaborn и hue.

Несколько линий из столбцов таблицы

```
weather.plot(x="day",
    y=["moscow", "kazan"], kind="line")
plt.xlabel("День")
plt.ylabel("Температура, °C")
plt.legend(["Москва", "Казань"])
plt.show()
```



Без kind по умолчанию тоже получились бы линии. Можно писать и сокращения: .plot.bar(), .plot.hist(), .plot.scatter(x=..., y=...).

Сохранение и самостоятельная работа

Теперь мы умеем брать сведения из списка, таблицы или файла и выбирать график по вопросу. Линия показывает изменение по упорядоченному аргументу; точки — связь двух чисел; столбцы — значения категорий; гистограмма — распределение.

pandas готовит данные и сводки, Seaborn помогает связывать визуальные свойства со столбцами, Matplotlib управляет самим рисунком. Способы можно сочетать: начать через pandas, а подписи и сохранение добавить через plt.

Сохраним таблицу и прочитаем обратно

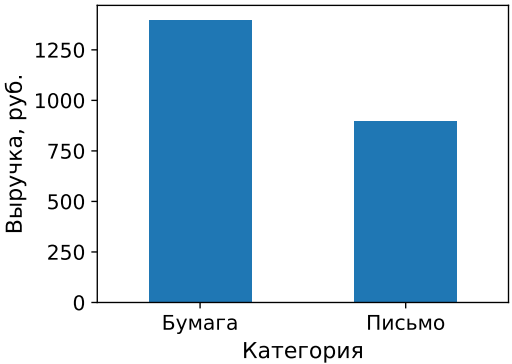
```
summary = revenue_by_category.reset_index(
    name="revenue"
)
summary.to_csv(
    OUTPUT / "sales_summary.csv",
    index=False)
pd.read_csv(OUTPUT / "sales_summary.csv")
```

	category	revenue
0	Бумага	1400
1	Письмо	900

Справа — результат обратного чтения сохранённого файла, а не вручную набранная копия. Папка OUTPUT настроена в начале ноутбука.

Сохранение рисунка не меняет его смысл

```
revenue_by_category.plot(
    kind="bar", rot=0)
plt.xlabel("Категория")
plt.ylabel("Выручка, руб.")
plt.savefig(OUTPUT / "sales.png", dpi=150,
            bbox_inches="tight")
plt.show()
```



Справа тот же рисунок, который записан в sales.png. Перед публикацией проверьте подписи, единицы измерения и смысл каждой высоты.

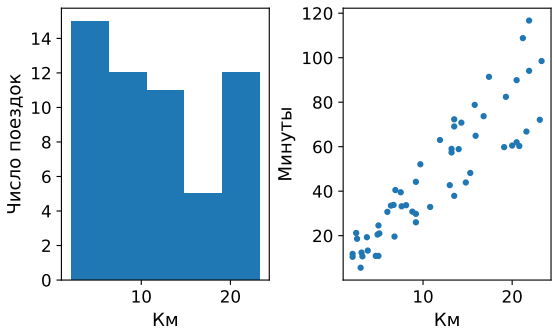
Дополнительно: несколько областей рисунка

Для одного рисунка достаточно разобранных команд plt. Если нужны две независимые области рядом, удобно получить явные переменные. fig обозначает весь рисунок, axes[0] и axes[1] — его области.

Это дополнительный способ управления, а не обязательный шаг перед каждым графиком. Методы областей похожи на знакомые функции: axes[0].hist вместо plt.hist, set_xlabel вместо xlabel.

Два вопроса на одном рисунке

```
fig, axes = plt.subplots(
    1, 2, figsize=(5.5, 3.3),
    layout="constrained")
axes[0].hist(trips["distance_km"], bins=5)
axes[0].set_xlabel("Км")
axes[0].set_ylabel("Число поездок")
axes[1].scatter(trips["distance_km"],
                trips["minutes"], s=12)
axes[1].set_xlabel("Км")
axes[1].set_ylabel("Минуты")
plt.show()
```



Здесь переменные областей действительно полезны: понятно, куда направлена каждая команда.

В обязательном ДЗ этот интерфейс не требуется.

Что сделать самостоятельно

Домашнее задание находится в hw/homework.ipynb. Первые четыре задания — графики по спискам: погода, растения и поездки. Затем — небольшая таблица растений, чтение CSV о поездках, группировка, встроенные графики pandas и Seaborn. Для обязательной части не нужны fig и ax. Перед сдачей перезапустите ядро, выполните все ячейки и сохраните ноутбук с рисунками и пояснениями. Matplotlib · pandas · Seaborn